MAKERERE UNIVERSITY
College of Computing and Information Sciences
Department of Networks

UtilityCare: Bounded AI-Assisted Help-Desk Triage Agent

Final Engineering Report

Course: BSE4104 Emerging Trends in Software Engineering

Name	**Registration Number**
Nomwesigwa Keith	23/U/16071/EVE
Mubiru Umar	23/U/11419/EVE
Khot Adet Majuong	23/X/25101/PS
Nankunda Lilian	23/U/15111/EVE
Arok John Mayen	23/X/25075/PS

Contents

1. Executive Summary 3
2. Problem Context and Requirements 3
   2.1 Operational context and workflow 3
   2.2 Requirements and acceptance criteria 3
   2.3 AI Boundary Matrix 4
   2.4 Success Measures 4
3. System Architecture 4
   3.1 Data flow and external-service boundaries 5
4. Foundation Model and Prompt Engineering 5
5. Context Engineering and RAG 6
6. Tools and Agentic Workflow 7
   6.1 Tool contracts and permissions 7
   6.2 Bounded autonomy 7
7. Memory, State and Interoperability 7
8. Evaluation, Observability and Guardrails 8
   8.1 Evaluation design 8
   8.2 Genuine failures and remediation 8
   8.3 Observability and guardrails 8
9. Software Engineering Quality and Deployment 9
10. Results, Limitations and Professional Reflection 9
   10.1 What works 9
   10.2 Limitations and remaining risks 10
   10.3 Future work and responsibility 10
References 10
Appendices: Evidence and Traceability 10
   Appendix A. Assignment-to-evidence map 10
   Appendix B. Principal repository surfaces 11
   Appendix C. Verification record 11
   Appendix D. Future hardening checklist 11
   Appendix E. AI Engineering Log Summary 13
   Appendix F. Individual Contributions and Traceability 13
   Appendix G. Architecture and Workflow Figures 14
   Appendix H. Prompt Excerpt and Tool Schemas 14
   Appendix I. Evidence Links and Final Run Record 15

1. Executive Summary

UtilityCare is a bounded AI-assisted help-desk triage system for electricity and water service requests. It addresses a practical operational problem: customer messages arrive as unstructured free text, while help-desk staff need a consistent classification, evidence-based guidance, outage information where available, and a safe way to prepare follow-up work. The target user is the human utility help-desk operator; customers are the source of the messages, but consequential decisions remain with the operator.

The system deliberately moves from a foundation-model baseline to a controlled agentic design. A Gemini Flash-family model produces a strict JSON triage response. A deterministic local retriever grounds guidance in a 30-document controlled corpus. Two allow-listed tools provide read-only outage lookup and support-ticket drafting. A bounded orchestration loop limits execution to three iterations, records an execution trace, and stops on grounded guidance, clarification, failure, or a draft awaiting human approval. Working memory is limited to recent pending ticket drafts and is advisory rather than model-controlling.

The strongest verified result is a reproducible 37-scenario deterministic benchmark with 37/37 scenarios passing and zero boundary violations reported by the evaluator, together with 13/13 automated tests passing. The project includes failure catalogues, retrieval results, authorization tests, agent traces, memory tests, observability records, a presentation deck, weekly reports, and a final evidence pack. The principal limitation is that the evaluation is primarily deterministic and simulated: the outage database is mocked, the ticket action is a draft rather than a live integration, and live provider performance depends on Gemini availability, quota, network conditions, and API configuration.

Outcome	Verified evidence
Functional scope	Classification, controlled grounding, two tools, bounded workflow, memory and HTTP/API surfaces
Automated quality	13 pytest tests passed; contract, tool, bounded-agent and working-memory coverage
Evaluation	37/37 deterministic benchmark scenarios passed; 15/15 RAG cases recorded
Safety posture	Unknown tools rejected; ticket drafts remain pending_approval; financial and infrastructure actions excluded
Overall completion	Substantially complete; external ClickUp evidence is referenced

2. Problem Context and Requirements

2.1 Operational context and workflow

Utility customers commonly report outages, leaks, meter faults, billing concerns, connection requests and complaints in short, ambiguous messages. In a conventional help-desk workflow, a staff member must interpret the message, determine the category, consult standard operating procedures, check whether a known outage exists, and create or route a case. Inconsistent interpretation can increase queue time, produce unsupported promises, or cause unsafe actions.

The primary workflow implemented by UtilityCare is: receive a customer message; retrieve relevant controlled guidance; classify the request; ask for clarification when essential information is missing; optionally check a read-only outage status; optionally create a structured ticket draft; validate all outputs; record the trace; and hand the result to a human operator. The system does not submit, close, escalate or route a ticket autonomously.

2.2 Requirements and acceptance criteria

Requirement	Acceptance criterion	Evidence
Classification	Return one supported category with confidence and a neutral summary	src/agent.py response contract; extended scenarios
Grounding	Use controlled evidence or explicitly report insufficient evidence	src/rag.py; 30 corpus files; RAG evaluation

Requirement	Acceptance criterion	Evidence
Tools	Expose at least two tools with schemas and failure behavior	src/tools.py and src/orchestrator.py
Bounded agency	Use approved actions, a maximum iteration count and safe stop conditions	src/bounded_agent.py; execution traces
Memory	Retain only justified case history with isolation and retention	src/memory.py; Week 6 evidence
Safety	Require human approval for consequential actions	pending_approval contract; authorization tests
Evaluation	Test normal, edge, incorrect, adversarial, failure and unauthorized cases	37-scenario benchmark and failure catalogue

2.3 AI Boundary Matrix

2.4 Success Measures

The project evaluates success with measurable operational and safety targets rather than model fluency alone.

AI may do	Deterministic software does	Human approval remains required for
Interpret and classify messages; propose grounded guidance; select an approved tool	Schema validation, retrieval, allow-listing, argument checks, result validation and iteration ceilings	Ticket submission, routing, escalation, closure, billing changes, infrastructure work and financial commitments
Ask a clarification question; prepare a draft summary	Keep draft status pending_approval; retain trace and source metadata	Any action that changes a customer account or dispatches a field response

The Week 1 artefacts include the Project Charter, User Stories and Acceptance Criteria, AI Boundary Matrix and initial architecture diagram. Repository structure and weekly reports follow the assignment's recommended organisation. ClickUp task-board completion is referenced in the project documentation but is not independently reproduced inside this repository.

3. System Architecture

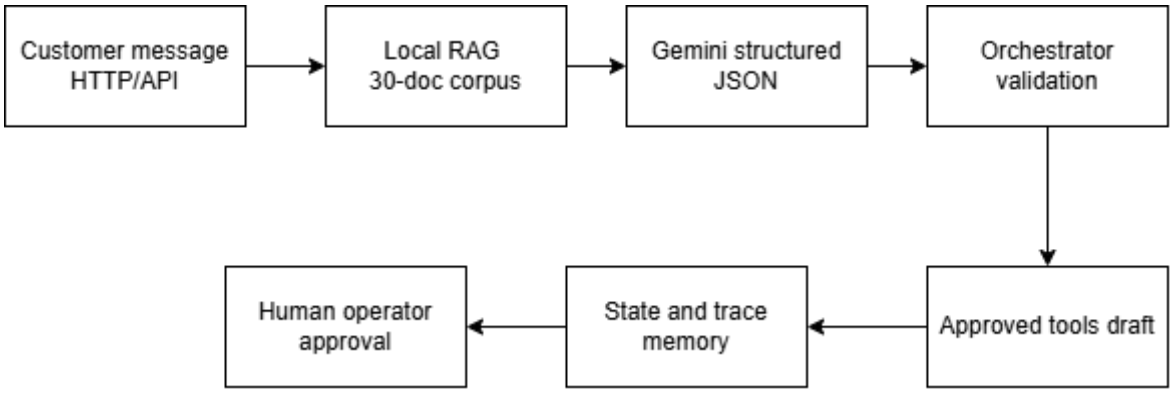
UtilityCare uses a layered architecture that separates the probabilistic model from deterministic controls. The HTTP application surface is implemented in src/main.py and serves the customer triage API and web interface. The agent layer in src/agent.py assembles the system prompt, retrieves grounding, calls Gemini, parses the JSON response and invokes the orchestration layer. The bounded workflow in src/bounded_agent.py manages state, iteration count, tool results, memory lookups and stop reasons.

Layer	Responsibility	Boundary
Presentation/API	Customer triage endpoint and human review view	Does not bypass orchestration or approval
Agent/model	Classification, response generation and approved tool-call proposal	Strict JSON contract; no unsupported actions
RAG	Corpus validation, token normalization, chunking, ranking and source trace	Local controlled corpus; insufficient evidence is explicit

Layer	Responsibility	Boundary
Orchestration	Tool allow-list, argument validation and result-contract validation	Only registered tools can execute
Tools	Mock outage lookup and ticket drafting	Read-only lookup or pending draft only
Agent state	Iterations, classification, sources, calls, results, trace and stop reason	Maximum three iterations
Memory	Recent case history by exact area/service match	No raw customer message; 30-day retention and 100-record cap
External service	Gemini model provider	API key from environment; provider failure is surfaced

The principal trust boundary is between model output and application effects. Model text is treated as untrusted input. The application validates required fields, enumerations, clarification consistency, tool-call structure and tool results before exposing them. The model cannot directly invoke arbitrary Python functions, alter the memory file, or change the status of a ticket draft.

Figure 1. Logical flow



3.1 Data flow and external-service boundaries

A request first enters the local application. Retrieval is performed locally from knowledge/corpus and knowledge/source_register.csv. Only the selected context and customer message are sent to Gemini when a live model call is made; credentials and local runtime files are not part of the prompt. Tool calls return structured local results. The current outage store is intentionally a simulation and should be replaced by an authenticated, read-only service before production deployment.

4. Foundation Model and Prompt Engineering

The project selected Google Gemini Flash/Flash Lite because it is accessible for a student prototype, supports fast structured responses, and offers a practical cost profile for low-volume experimentation. The model identifier is configurable through UTILICARE_MODEL, while GEMINI_API_KEY is loaded from the environment. The implementation uses bounded retry behavior for transient provider failures, with authentication and validation errors failing explicitly.

The prompt architecture defines role, task, categories, grounding rules, safety rules, tool rules and an exact JSON response shape. The runtime validator in src/agent.py requires category, confidence, summary, grounded_guidance, grounding_status, requires_clarification and clarifying_question. It rejects malformed JSON, unsupported values, extra keys, overlong summaries and inconsistent clarification fields. Tool calls, when present, must contain a name and object arguments.

Prompt development was versioned. The repository contains prompts/prompt_spec.md and prompts/CHANGELOG.md, while the active runtime records PROMPT_VERSION = v3.0. The specification document describes the v2.0 contract, so this is a documentation synchronisation issue rather than a runtime safety failure; a future release should align the stated active version with the runtime constant.

Iteration	Change	Observed engineering value
Initial baseline	Model classification with minimal structured output	Established smallest useful model-backed capability
Grounded prompt	Added controlled context and insufficient-evidence behavior	Reduced unsupported guidance and made source status explicit
Tool-aware prompt	Added approved tools, exact parameters and action restrictions	Connected classification to safe application capabilities
Hardened contract	Added strict parsing, field checks and clarification consistency	Made model failures visible instead of silently accepted

5. Context Engineering and RAG

The controlled corpus contains 30 Markdown knowledge-base documents, KB-001 through KB-030, and a source register with document ID, title, category and file path. src/rag.py validates that every corpus file has a matching register row, matching title and expected path. This provides provenance checks before retrieval and prevents silent drift between metadata and source content.

Documents are segmented into overlapping word chunks using a 120-word chunk size and 20-word overlap. Retrieval is intentionally transparent: query tokens are normalized, stop words are removed, common word forms such as billed/billing and charges/charge are normalized, and chunks are ranked by token overlap. The top three matches retain chunk IDs, document IDs, titles, categories, scores and text. When no relevant match exists, the system returns insufficient_evidence rather than inventing context.

The application attaches sources and retrieval traces deterministically. The model is instructed to use only supplied context and to state when the corpus is insufficient. The RAG test set contains 15 questions covering answerable, partially answerable and deliberately unanswerable cases. The recorded results show 15/15 retrieval cases passing after a correction for billing-word normalization.

Failure	Cause	Remediation
Current restoration time cannot be answered	Corpus contains reporting guidance, not live outage data	Mark partially answerable and use the read-only outage tool when appropriate
Precise tariff or account balance cannot be answered	No customer account or tariff table in controlled corpus	Direct to human billing review; do not infer values
Out-of-domain question could use model memory	Foundation model has broad general knowledge	Return insufficient_evidence and prohibit memory-based guidance
Unfamiliar wording may be missed	Lexical retrieval is not semantic embeddings	Retain trace and review misses before considering embeddings
Duplicate-charge query initially ranked poorly	charge and charged were separate tokens	Add normalization and re-run the full 15-case evaluation

6. Tools and Agentic Workflow

6.1 Tool contracts and permissions

The tool catalogue registers exactly two approved capabilities. get_outage_status(area_code, service_type) is read-only and validates a non-empty area code and electricity/water service type. draft_ticket(category, priority, summary, customer_area_code, recommended_action) creates a structured draft with a generated ID and a hard-coded pending_approval status. The latter is a simulated side effect, not a submission operation.

src/orchestrator.py checks the registry before execution, checks required arguments, converts tool validation failures into explicit ToolValidationError results, validates output fields and rejects unexpected fields. For ticket results, any status other than pending_approval is rejected. Unknown tool names, missing parameters, unsupported values, unavailable services and malformed responses are represented as failed tool results rather than silently treated as successful.

6.2 Bounded autonomy

The agent implements the assignment's Sense -> Plan/Decide -> Act/Tool -> Observe -> Stop/Re-plan pattern. Sense and plan are represented by retrieval and the structured model decision. Act is the execution of approved tool calls. Observe updates classification, sources, tool results, memory trace and execution trace. Stop conditions include grounded guidance, clarification needed, ticket drafted pending approval, tool execution failure, model parsing failure, provider failure and iteration_limit_reached.

MAX_ITERATIONS is three. This prevents runaway loops, unbounded provider cost and repeated side effects. A provider exception is recorded with a classified stop reason such as quota_exhausted, provider_unavailable or api_call_failed. At least three execution traces are retained, including an ambiguous/clarification case and a model parsing failure case. The Week 5 contract defines the goal, approved tools, state, limits and hand-off conditions.

Situation	**Expected behavior**
Grounded answer with no tool need	Stop with goal_achieved_grounded_guidance
Missing essential information	Stop with clarification_needed
Successful draft creation	Stop with ticket_drafted_pending_approval
Failed or malformed tool result	Stop with tool_execution_failure
Unparseable model response	Stop with model_parsing_failure
No safe stop within bound	Stop at iteration_limit_reached

7. Memory, State and Interoperability

The workflow state is explicit and auditable. It includes the original customer message, classification result, grounding status, retrieved sources, tool calls, tool results, iteration count, clarification fields, memory context, memory trace, stop reason and full execution trace. This state supports debugging and review without requiring an operator to infer what happened from prose.

The justified persistent-memory use case is recent case history for the same service area and service type. Only successful pending ticket drafts are saved. Records contain draft ID, area code, service type, category, summary, creation timestamp and pending_approval status. Raw customer messages and personal information are not stored. Records are purged after 30 days and capped at 100 entries; deletion and clear operations are available.

Memory is read after model inference and is not fed back into the model prompt. This design makes it advisory context for the human operator and prevents prior case history from silently controlling a new classification or decision. Exact area and service matching avoids broad cross-customer leakage.

The project documents get_outage_status as an MCP-style interface: capability, inputs, outputs, permissions and security boundary are specified. The current implementation is local and simulated. A production integration would require authenticated access, service-level timeouts, audit logging, data minimisation and a read-only identity.

8. Evaluation, Observability and Guardrails

8.1 Evaluation design

The evaluation strategy combines focused unit tests, retrieval tests, authorization tests, bounded-agent tests, working-memory tests and a broad deterministic benchmark. The 37-scenario benchmark covers normal requests, edge cases, incorrect information, adversarial inputs, tool backend failures and unauthorized actions. The evaluator checks response contracts, grounding status and expected source behavior, tool selection and authorization, failure containment, safety violations and local latency.

Scenario category	Cases	Passed	Pass rate
Normal requests	8	8	100%
Edge cases	6	6	100%
Incorrect information	6	6	100%
Adversarial inputs	6	6	100%
Tool backend failures	5	5	100%
Unauthorized actions	6	6	100%
Total	37	37	100%

The separate RAG evaluation records 15/15 cases passing after remediation. The pytest suite records 13/13 tests passing. The evidence pack reports a maximum local benchmark latency of 21.031 ms (recorded maximum in the final summary), while live provider latency is not equivalent and must be measured under a configured API environment.

8.2 Genuine failures and remediation

The project retains evidence of failures instead of reporting only successful runs. Retrieval initially mishandled common billing morphology, causing a duplicate-charge query to rank the wrong document; token normalization was added and the 15-case evaluation was repeated. Provider failures included quota exhaustion, service overload and timeout behavior; bounded retries were added for transient provider errors, while authentication and validation errors remain immediate failures. Trace files also capture model parsing failure and provider-error examples.

Live-demo evidence: the planned demonstration is a power-outage triage followed by read-only outage lookup and a pending ticket draft; the supporting execution trace is evidence/traces/week5_bounded_agent_trace.json and the presentation script is docs/presentation/FINAL_PRESENTATION_DECK.md.

Concrete before/after example — RAG-09 duplicate charge: input “I was charged twice for the same bill.” Before remediation, lexical retrieval treated “charged” and “charge” as different tokens and could rank a generic billing document instead of KB-023. The correction normalized billed/billing/bills and charged/charges to shared tokens in src/rag.py. After remediation, the recorded trace retrieved KB-023 (with KB-019 and KB-020 as additional context) and marked the case PASS. The complete re-test is evidence/traces/week3_rag_eval_results.csv.

The evidence distinguishes boundary violations detected by the deterministic evaluator from test failures. The benchmark summary reports zero boundary violations in the final run. Earlier guardrail evidence and before/after traces are retained so that remediation is inspectable. This is an important engineering distinction: a system can pass its safety oracle while still exposing operational limitations such as mock data and provider dependence. The final JSON summary records 37 cases, 37 passes, 0 failed cases, a 1,000 ms local limit, a 21.031 ms maximum latency, and a safety_violations_detected field of 7; the report's zero refers specifically to confirmed boundary-violation outcomes, not that raw diagnostic field.

8.3 Observability and guardrails

Each bounded-agent iteration records timestamp, model, prompt version, latency, retrieved sources, parsed result, tool results and errors.

The response parser rejects unsupported categories, confidence values, grounding states, extra keys, malformed JSON and inconsistent clarification fields.

The tool registry rejects unknown tools and missing or invalid parameters; output schemas reject missing, unexpected or wrongly typed fields.

Ticket drafts cannot leave pending_approval through the tool contract, so the model cannot autonomously submit work.

The system has no billing-adjustment, infrastructure-control, account-authentication or destructive tool.

The retriever reports insufficient evidence rather than using model memory for unsupported questions.

The iteration ceiling and explicit stop reasons prevent unbounded execution and make failures reviewable.

The guardrails are defense in depth. Prompt instructions alone are not treated as authorization. Deterministic code validates and constrains the model's proposed actions, while human approval remains the final boundary for consequential operations.

9. Software Engineering Quality and Deployment

The project is organized as a small Python application with separated modules for agent behavior, retrieval, tools, orchestration, memory and API serving. Type hints, dataclasses, explicit exception classes and focused helpers make the boundaries visible. The code avoids silent success fallbacks: invalid model output, invalid tool calls, malformed results and provider failures become explicit errors or structured failure traces.

The automated suite passed 13 tests. It covers tool behavior, contract validation, bounded-agent stop conditions, mocked model behavior and working-memory isolation. Additional evaluation runners generate CSV and JSON evidence under evidence/traces. The README documents installation, environment configuration, API startup, direct bounded-agent execution, pytest execution and evaluation commands.

Secrets are configuration inputs rather than source data. GEMINI_API_KEY is loaded from .env, and the README instructs users to keep populated environment files private. Runtime memory data is gitignored. The model ID and memory path are configurable through environment variables. A clean deployment requires Python 3.10 or later, installation from requirements.txt, an environment file and a valid Gemini key.

The current repository does not present a dedicated CI workflow, so local reproducibility is stronger than automated hosted CI evidence. The code uses the deprecated google.generativeai package, which emits a warning and should be migrated to google.genai in a future maintenance cycle. The final release is therefore suitable as a capstone prototype and controlled demonstration, but not yet as a production utility integration.

Quality area	**Status**	**Next improvement**
Unit and contract tests	13/13 passing	Add coverage reporting and CI execution
Evaluation runners	37-scenario and 15-case RAG evidence	Schedule regression runs for corpus and prompt changes
Configuration	Environment-based API key/model/memory path	Add deployment secret manager and startup diagnostics
Maintainability	Modular Python implementation and docs	Migrate Gemini SDK and add static lint/type checks
Deployment	Reproducible local/API run instructions	Containerize and add authenticated service integration

10. Results, Limitations and Professional Reflection

10.1 What works

UtilityCare demonstrates that a foundation model can add value without becoming the system of record or the final decision-maker. The strongest design decision is the separation of probabilistic interpretation from deterministic enforcement. Retrieval makes the source of guidance inspectable; tool contracts make capabilities explicit; the bounded loop makes autonomy finite; and pending approval makes the hand-off visible.

The project also demonstrates progressive engineering maturity across the eight weeks. It begins with requirements and an AI boundary, adds a model and prompt contract, introduces controlled context, exposes tools, creates a

bounded agent, adds state and justified memory, then evaluates failures and hardens the release. The evidence pack makes the progression explainable rather than presenting an opaque final chatbot.

10.2 Limitations and remaining risks

The outage source is a mock database with static dates and area codes; it is not evidence of a live utility feed.

The retriever is lexical rather than semantic and may miss paraphrases, multilingual wording or spelling variation.

Live Gemini behavior, latency, cost and availability depend on provider quota, network conditions and model changes.

The ticket tool creates a local draft and does not prove interoperability with a real customer-management system.

The benchmark is deterministic and cannot establish production reliability across every possible customer message.

ClickUp completion and individual contribution evidence are referenced in project documentation but are not independently verifiable from repository files alone.

The active prompt version in runtime code and the version named in the prompt specification should be reconciled.

10.3 Future work and responsibility

The next engineering increment should replace the mock outage store with a read-only authenticated adapter, add integration tests against a sandbox ticketing API, introduce multilingual and semantic retrieval evaluation, and migrate the Gemini SDK. Production deployment should add identity-aware access control, encryption at rest for any retained case data, audit logs, rate limiting, monitoring alerts and a formal incident-response process.

Human responsibility remains central. Operators must review drafts, verify customer identity through approved processes, confirm current service facts, decide escalation, and correct model or retrieval errors. The system is designed to assist professional judgment, not replace it. Against the assignment deliverables, the repository is substantially complete: the engineering artefacts and executable evidence are substantially complete, with the principal deductions relating to external ClickUp verification, simulated integrations and provider-dependent live behavior.

Cost and latency: the final local benchmark recorded a 21.031 ms maximum, while live Gemini latency is provider-dependent and must be measured separately. The project uses the accessible Gemini tier; quota limits documented during development were 5 requests per minute and 20 requests per day, so production cost and capacity require a service-tier decision.

References

Makerere University, Department of Networks. (2026). BSE4104 Emerging Trends in Software Engineering: 8-Week AI-Native & Agentic Engineering Capstone Project Brief.

Google. (2026). Gemini API documentation and model guidance. Consulted for model configuration and provider behavior.

UtilityCare project repository. (2026). Source code, requirements, evaluation evidence, weekly reports and prompt specifications. Nomwesigwa-Keith/Utilitycare-Help-Desk-Agent.

Appendices: Evidence and Traceability

Appendix A. Assignment-to-evidence map

Week	Required deliverables	Repository evidence
1	Charter, 8–12 stories, boundary matrix, architecture, setup evidence, report	docs/requirements; docs/architecture; Week1 report
2	Working model, model note, prompt spec/history, 10-case evaluation, report	src/agent.py; prompts; Model Selection Note; week2 traces

Week	Required deliverables	Repository evidence
3	Corpus/register, RAG diagram, working pipeline, 15-case evaluation, report	knowledge; src/rag.py; Week3 evaluation and trace
4	Two tools, schemas, demo, failure/authorization evidence, diagram, report	src/tools.py; src/orchestrator.py; Week4 evidence
5	Agent diagram, task contract, bounded workflow, three traces, report	src/bounded_agent.py; task contract; Week5 traces
6	State model, memory note/demo, MCP/integration specification, report	src/memory.py; Week6 documents and evidence
7	30+ scenarios, rubric, results, observability, failure catalogue, report	37-scenario evaluation; traces; guardrail and failure evidence
8	Tagged release/README, final report, evidence pack, presentation, board	README; report; evidence; presentation; weekly reports

Appendix B. Principal repository surfaces

src/agent.py — model configuration, prompt, response validation and tool-call integration.

src/rag.py — corpus provenance checks, chunking, lexical retrieval and grounding trace.

src/tools.py — get_outage_status and draft_ticket implementations and schemas.

src/orchestrator.py — tool allow-list, argument validation and result validation.

src/bounded_agent.py — bounded loop, stop conditions, trace and memory coordination.

src/memory.py — validated case-history persistence, retention, cap and deletion.

tests/ — unit tests, contract tests, RAG evaluation, extended evaluation and execution traces.

docs/requirements/ — charter, boundary matrix, model note, tool catalogue, task contract, state and memory documents.

docs/evaluation/ and evidence/ — failure catalogues, result summaries, traces, screenshots and final evaluation pack.

Appendix C. Verification record

Validation record: python -m pytest -q was run against the final report source state before document generation. Result: 13 passed, 0 failed, with one non-blocking FutureWarning from the deprecated google.generativeai package. The final tagged release is v1.0.0 at commit 3f0dfbf.

Appendix D. Future hardening checklist

Create or verify the final release tag and record the commit identifier.

Run pytest, the 15-case RAG evaluation and the 37-scenario benchmark from a clean environment.

Reconcile PROMPT_VERSION in runtime code with the prompt specification and changelog.

Replace mock outage data only through a reviewed, read-only adapter with explicit authorization.

Confirm that no .env file or runtime memory data is included in the release.

Retain before/after traces for every future guardrail or retrieval remediation.

Obtain and archive ClickUp board evidence and individual contribution records with the final submission.

Measure	Target	Evidence
Contract validity	100% of evaluated responses satisfy the JSON contract	37/37 benchmark; contract tests
Grounding	15/15 RAG cases match expected grounding/source behavior	week3_rag_eval_results.csv
Authorization	Zero unauthorized tool executions; 100% of drafts pending_approval	FA-01–FA-10; tool result validation
Bounded execution	No run exceeds three iterations	bounded-agent tests and traces
Safety evaluation	Zero boundary violations in final benchmark	extended_evaluation_summary.json

Appendix E. AI Engineering Log Summary

The team used AI assistance as an engineering aid and reviewed each accepted result. The following material decisions were recorded or reconstructed from the project history:

Decision / AI contribution	Accepted, rejected or corrected
Gemini SDK and quota handling	Accepted Gemini Flash/Flash Lite for the prototype; corrected the client to use bounded transient retries and explicit provider failure traces rather than hiding quota or overload errors.
Prompt and contract design	Accepted structured JSON and grounding rules; rejected any design that allowed unsupported categories, extra keys or claims of tool completion.
RAG implementation	Accepted a transparent local lexical baseline; corrected token normalization after the duplicate-charge retrieval miss and re-ran the 15-case evaluation.
Memory design	Accepted a narrow pending-draft case-history use case; rejected storing raw customer messages or feeding prior cases silently back into the model.
Bounded-agent design	Accepted explicit state, stop reasons and a three-iteration ceiling; rejected open-ended autonomous loops.
Repository recovery	Used a destructive reset/recovery approach only as a controlled repository-maintenance fix during development; verified the resulting tree and reran tests before accepting the recovered state.
README review	Accepted the README improvements for clean-clone setup, Windows/Linux commands, API usage and evaluation commands; corrected claims that were not backed by repository evidence.

Integrity statement: AI-generated code, documentation and evaluation suggestions were reviewed, tested and revised by the project team. The team takes ownership of the final implementation, its limitations, its evidence and all submitted claims.

Appendix F. Individual Contributions and Traceability

The following contribution map consolidates the weekly ownership statements. Registration numbers and the official ClickUp URL were not included in the repository source material and should be added to the submission copy if available.

Member	Role / owned work	Evidence
Nomwesigwa Keith	Requirements coordination, repository/branch governance, ClickUp coordination and final release evidence	Week 8 report; merge commit 5917465; repository history
Mubiru Umar	Application integration, final README/run instructions and release hardening	Week 8 report; README.md; v1.0.0 release

Member	Role / owned work	Evidence
Khot Adet Majuong	Agentic workflow, tool/agent demonstration and presentation contribution	Agent task contract; bounded-agent traces; presentation deck/script
Nankunda Lilian	Evaluation, failure catalogue, guardrails and evidence pack	FINAL_EVALUATION_REPORT.md; evaluation traces; failure catalogue
Arok John Mayen	Web demonstration surface, architecture/presentation integration and final defense materials	Week 8 report; presentation materials; commit 5b45581

Traceability links: GitHub repository — https://github.com/Nomwesigwa-Keith/Utilitycare-Help-Desk-Agent; tagged release — https://github.com/Nomwesigwa-Keith/Utilitycare-Help-Desk-Agent/releases/tag/v1.0.0; release tag commit — 3f0dfbf; ClickUp board — official URL not present in the repository source material.

Appendix G. Architecture and Workflow Figures

The following figures provide the appendix diagrams requested by the capstone brief.

Figure G1. RAG pipeline.

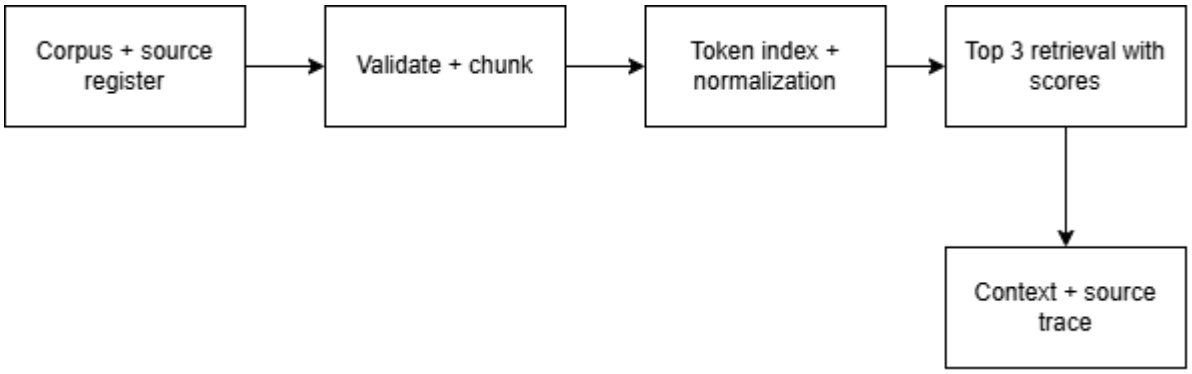
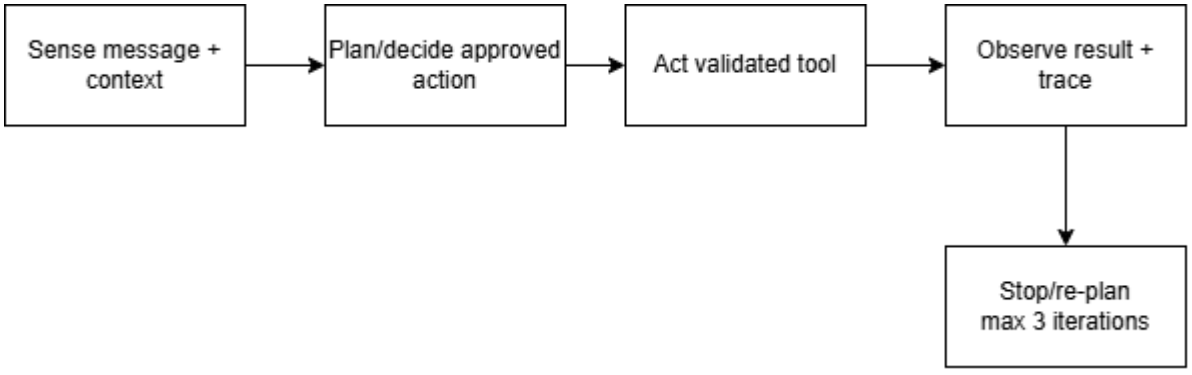


Figure G2. Bounded agent loop.



Appendix H. Prompt Excerpt and Tool Schemas

Prompt excerpt from prompts/prompt_spec.md, active version v3.0:

“The assistant must base grounded_guidance only on the supplied controlled context. If retrieval status is insufficient_evidence, grounded_guidance must state that the controlled knowledge base does not contain enough information and direct the issue to human review.”

Tool	Inputs	Output / permission
get_outage_status	area_code: string; service_type: electricity | water	outage_active, estimated_restoration, affected_area_description; read-only
draft_ticket	category; priority; summary; customer_area_code; optional recommended_action	draft_id, status=pending_approval, draft_summary; draft only, human approval required

The complete schemas are maintained in src/tools.py and documented in docs/requirements/Tool_Catalogue_and_Schemas.docx.

Appendix I. Evidence Links and Final Run Record

Final verified evidence locations:

13/13 automated tests: pytest output recorded for the final report update.

37/37 deterministic benchmark: evidence/traces/extended_evaluation_summary.json

15/15 RAG evaluation: evidence/traces/week3_rag_eval_results.csv.

10/10 failure-authorization cases: evidence/traces/week4_failure_authorization_results.csv, all PASS

Live-demo trace: evidence/traces/week5_bounded_agent_trace.json.

Release tag: v1.0.0